

`countofsquares` records the total number of squares that have been completely drawn in the game so far.

Now we can write a short function to calculate the score of a game. The score will be recorded as a list with two elements, like `[2,1]`. The first element of the score list represents the score of the first player, and the second element represents the score of the second player. [Listing 10-3](#) has our scoring function.

```
def score(game):
    score = [0,0]
    progress = []
    squares = 0
    for line in game:
        progress.append(line)
        newsquares = squarefinder(progress)
        if newsquares > squares:
            if len(progress)%2 == 0:
                score[1] = score[1] + 1
            else:
                score[0] = score[0] + 1
        squares=newsquares
    return(score)
```

[Listing 10-3](#): A function that finds the score of an in-progress dots and boxes game

Our scoring function proceeds through every line segment in a game in order, and considers the partial game consisting of every line drawn up to that turn. If the total number of squares drawn in a partial game is higher than the number of squares that had been drawn one turn previously, then we know that the player whose turn it was scored that turn, and we increment their score by 1. You can run `print(score(game))` to see the score of the game illustrated in [Figure 10-2](#).

Game Trees and How to Win a Game

Now that you've seen how to draw and score dots and boxes, let's consider how to win it. You may not be particularly interested in dots and boxes as a game, but the way to win at it is the same as the way to win at chess or